IMEN891K / IDSC501 / AXGS-501 · DATA ENGINEERING · FALL 2026

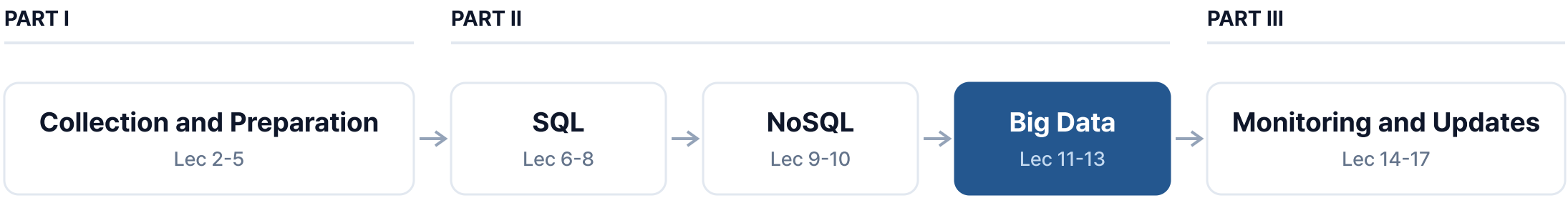
11.

Big Data I: Batch Processing

Sangwoo Mo · POSTECH

Industrial & Management Engineering

Where We Are


PART I
PART II
PART III
Collection and Preparation
Lec 2-5
SQL
Lec 6-8
NoSQL
Lec 9-10
Big Data
Lec 11-13
Monitoring and Updates
Lec 14-17

Outline: Big Data I

01	**Processing on Many Machines**	split a large job across machines
02	**How Spark Runs a Job**	understand when and where work runs
03	**Working with Spark**	query data and save the results

01 · Processing on Many Machines

01	**Processing on Many Machines**	split a large job across machines
02	**How Spark Runs a Job**	understand when and where work runs
03	**Working with Spark**	query data and save the results

Why Many Machines

Reading 100 TB from one disk takes days; splitting the read takes minutes

One machine



100 TB ÷ 200 MB/s ≈ **6 days**

1,000 machines

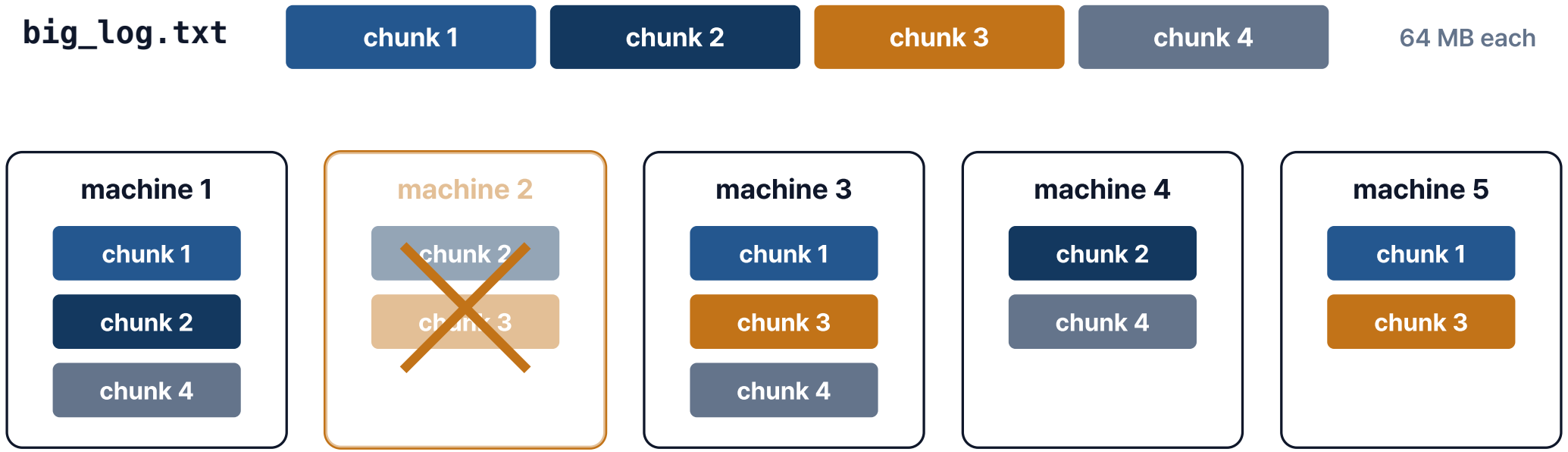


100 GB ÷ 200 MB/s ≈ **8 minutes**

Split the data, and each machine **reads only its own share**

Distributed File System (DFS)

A large file is cut into chunks, and each chunk is stored on several machines

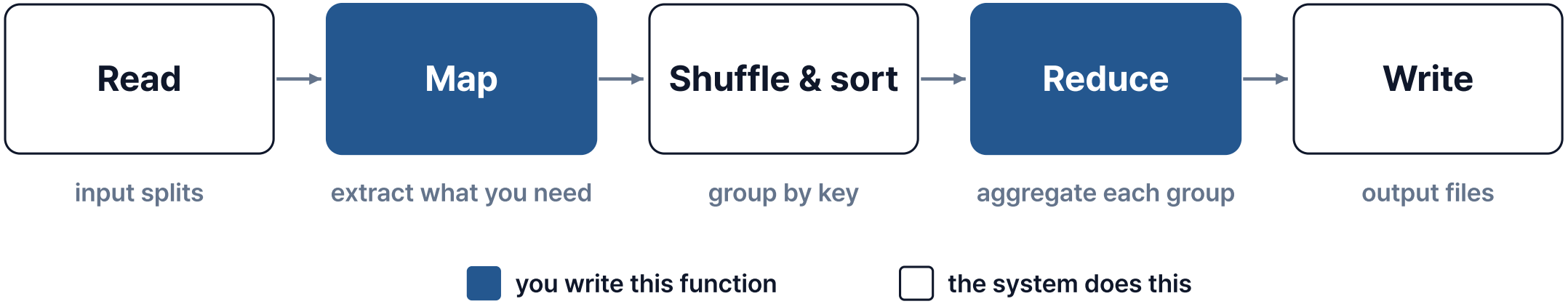


Every chunk has **3 copies** on different machines; losing machine 2 loses no data

Replicas on different machines: **one failure loses no data**

MapReduce in Five Steps

Google 2004, open-source Hadoop: every job has the same shape



Change the two **blue functions** to solve a different problem

Case study: Google's Web Index

Google rewrote its production indexing system for web search on MapReduce

> 20 TB

raw crawled documents, input to the indexer

5 to 10

MapReduce jobs in a row build the index

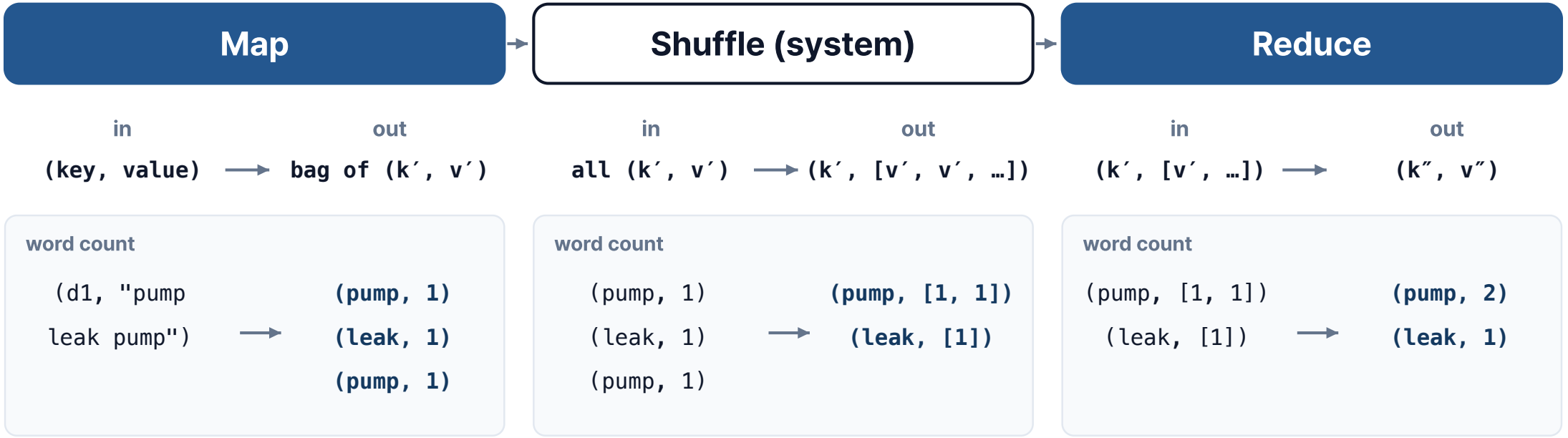
3,800 → 700

lines of C++ for one indexing phase

MapReduce moved **distribution and recovery** code out of the indexing application

Data Model: Key-Value Pairs

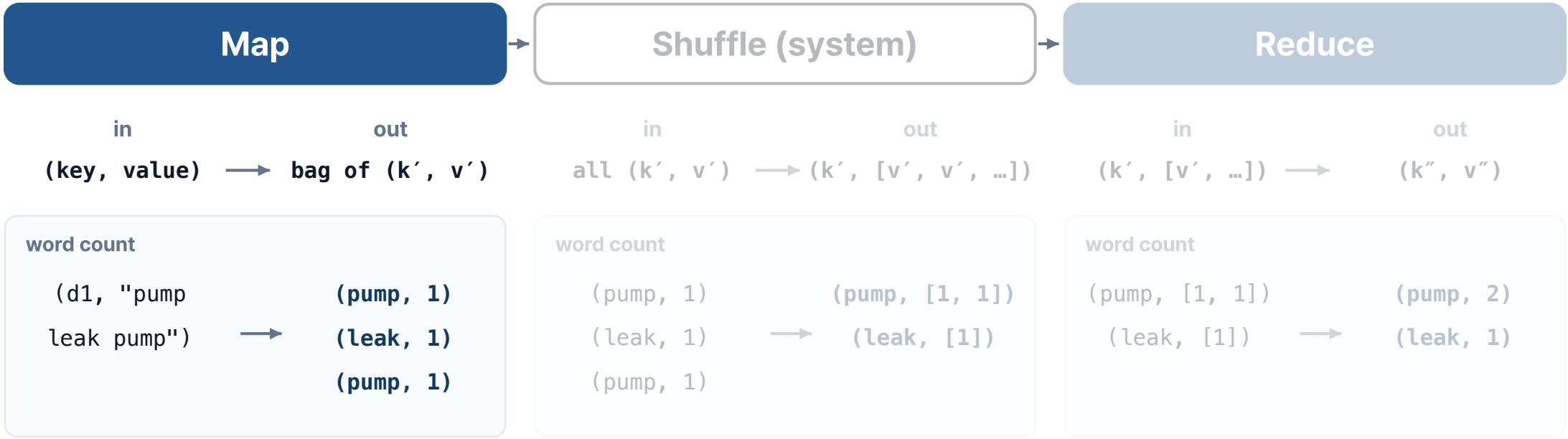
Every phase reads and writes bags of (key, value) pairs



One word-count record followed through **all three phases**

Step 1: Map

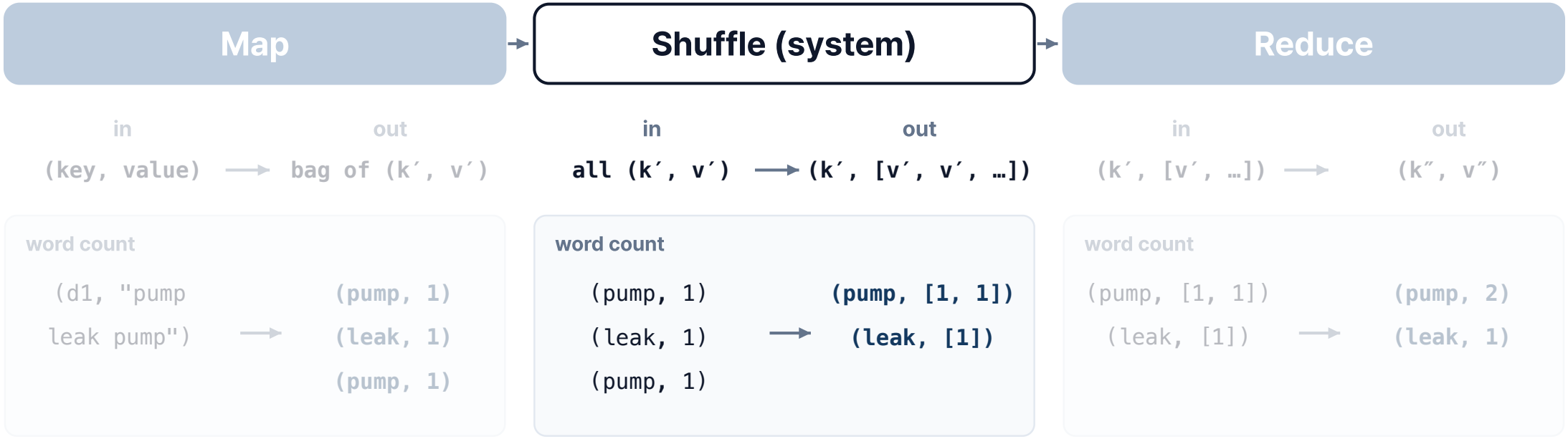
You write a function from one input pair to any number of new pairs



No pair depends on another, so **every split maps in parallel**

Step 2: Shuffle

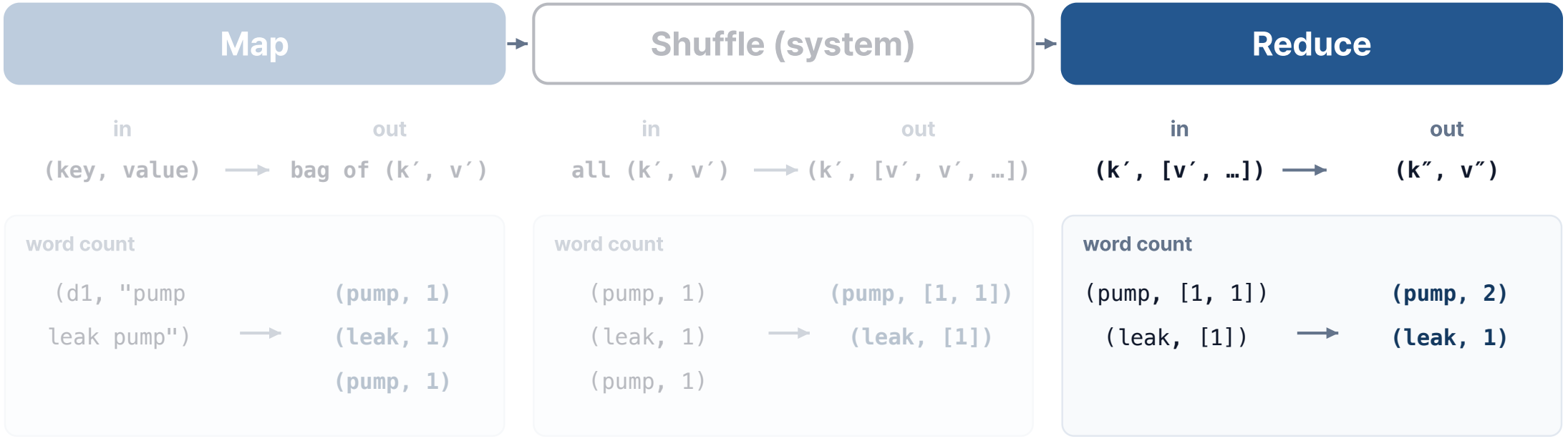
The system brings every value of one key to the same place



Grouping by key is the **network-heavy** step

Step 3: Reduce

You write a function from one key and its values to the result



Each key is reduced **independently**, in parallel

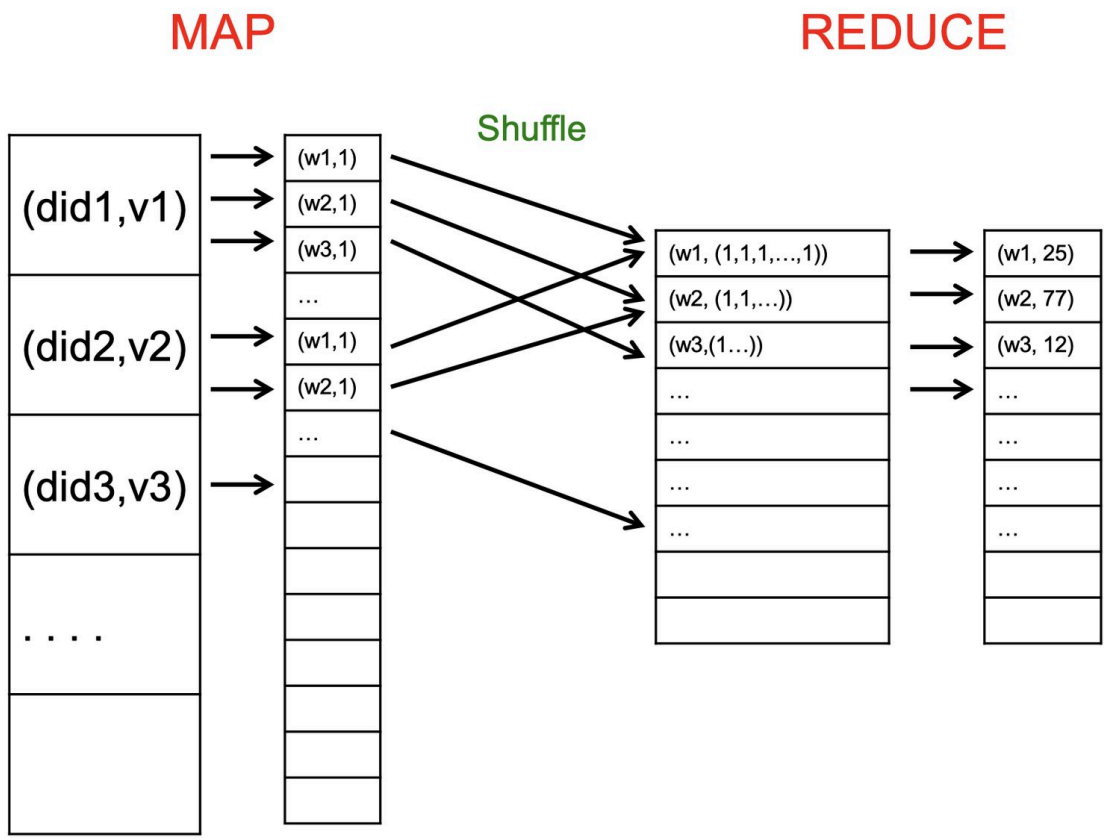
Word Count in Code

- Counting the number of occurrences of each word in a large collection of documents
- Each Document
  - The key = document id (did)
  - The value = set of words (word)

```
map(String key, String value):
  // key: document name
  // value: document contents
  for each word w in value:
    emitIntermediate(w, "1");
```

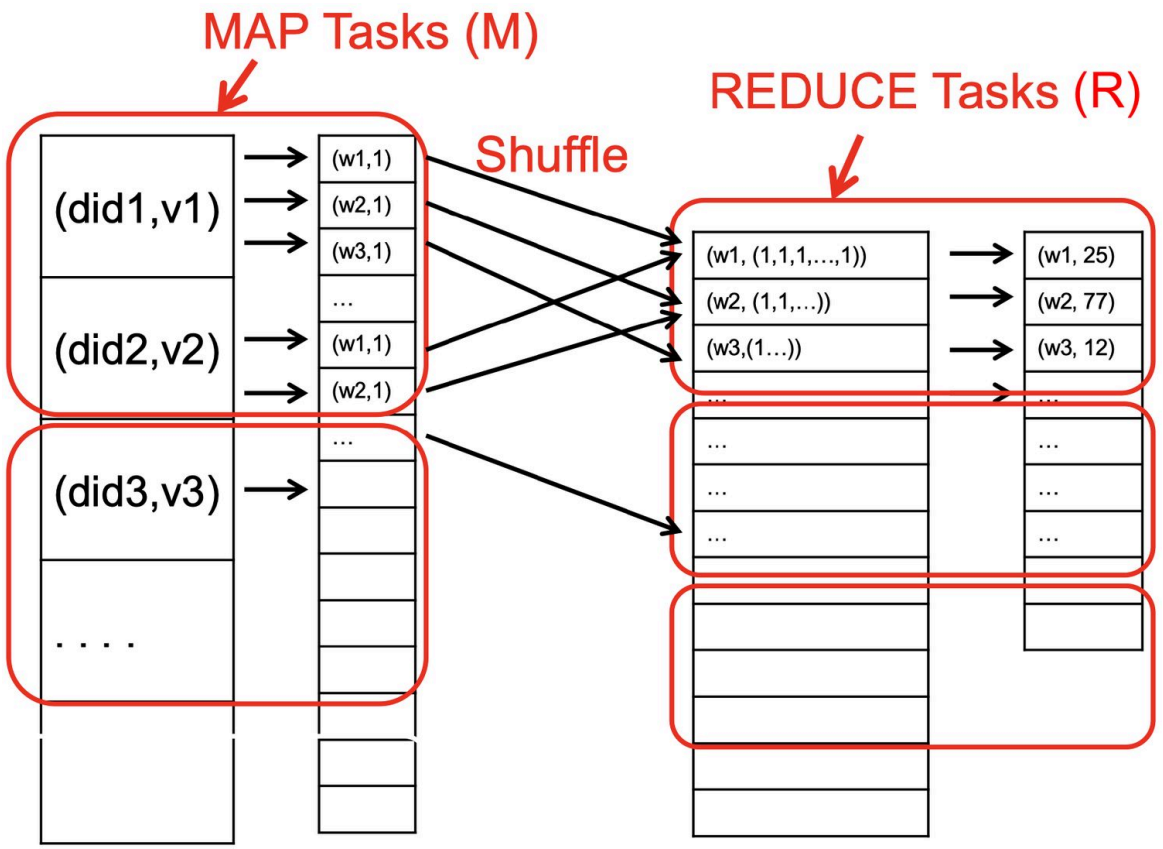
```
reduce(String key, Iterator values):
  // key: a word
  // values: a list of counts
  int result = 0;
  for each v in values:
    result += ParseInt(v);
  emit(AsString(result));
```

Example: The Data Flow



Map emits pairs, **shuffle groups them by key**, reduce folds each group

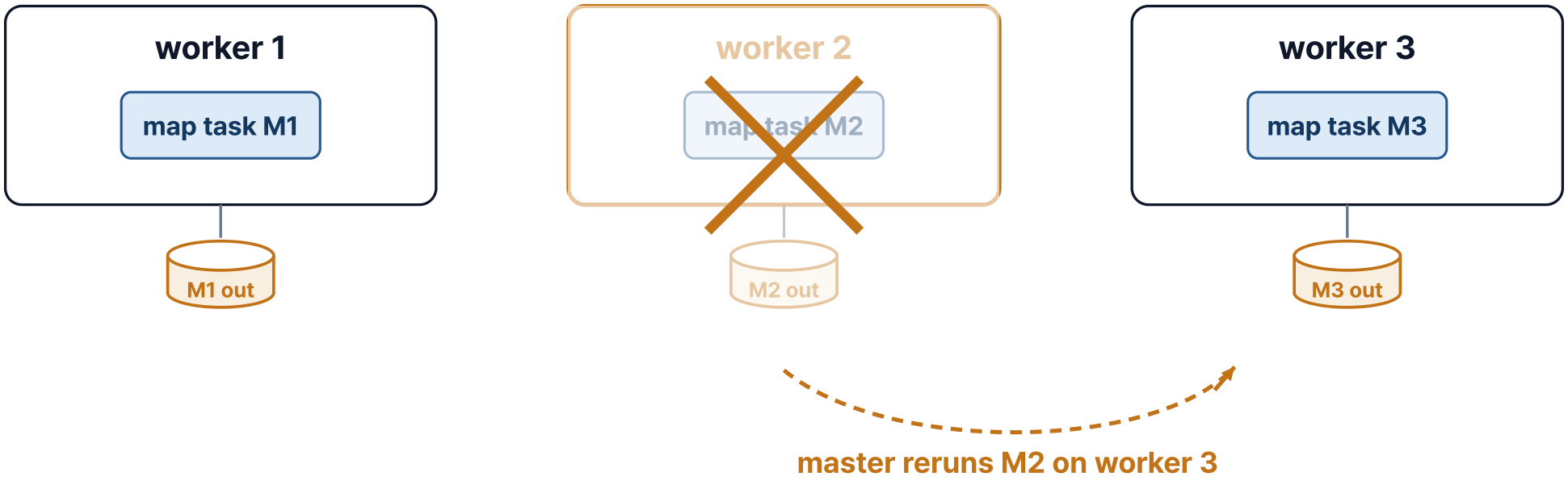
Map Tasks and Reduce Tasks



M map tasks and R reduce tasks, usually **many more than there are workers**

Fault Tolerance

With 10,000 servers, one fails about every hour on average



Map output sits on local disk; finished tasks are not redone

A failure costs **one task**, not the whole job

Demo: MapReduce Word Count

Reducer load is measured by input-value count, not elapsed time

1. Vary **R** from 1 to 4 and watch the largest load
2. Turn on the **hot key** and vary R again

Reducers 1 2 **3** 4 ☐ **Hot key:** `alarm` **in every log**

◀ Back | Step ▶ | Run all | 3 · Reduce

Input splits

SPLIT 1
d1 pump leak valve pump
d2 motor heat pump

SPLIT 2
d3 valve motor belt
d4 belt pump heat valve

SPLIT 3
d5 motor valve leak
d6 heat belt motor pump motor

Map output *(word, 1)*

MAP 1
pump,1 R2 | leak,1 R1 | valve,1 R1 | pump,1 R2 | motor,1 R1 | heat,1 R3 | pump,1 R2

MAP 2
valve,1 R1 | motor,1 R1 | belt,1 R2 | belt,1 R2 | pump,1 R2 | heat,1 R3 | valve,1 R1

MAP 3
motor,1 R1 | valve,1 R1 | leak,1 R1 | heat,1 R3 | belt,1 R2 | motor,1 R1 | pump,1 R2 | motor,1 R1

Shuffle by `hash(word) mod R` → reduce *(sum)*

REDUCER 1
motor ■■■■■ 5
valve ■■■■ 4
leak ■■ 2
11 values · largest load

REDUCER 2
pump ■■■■■ 5
belt ■■■ 3
8 values

REDUCER 3
heat ■■■ 3
3 values

Largest load: **11 values** (even split: 8)

Answer: MapReduce Word Count

A hot key creates a load floor that additional reducers cannot remove

reducers R	max load, no hot key	max load, hot key	even split, hot key
1	22	46	46
2	13	33	23
3	11	27	16
4	8	29	12

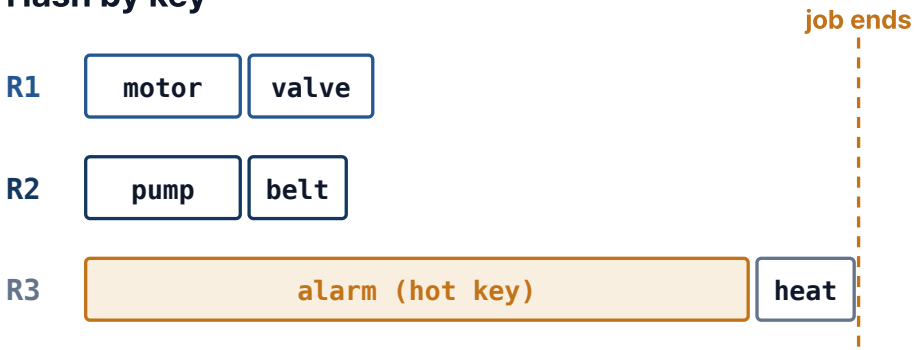
Hashing sends **all values of one key to one reducer**, so the reducer with `alarm` reads at least 24 values for any R

With the hot key, **R = 4 carries more load than R = 3**

Caveat: Data Skew

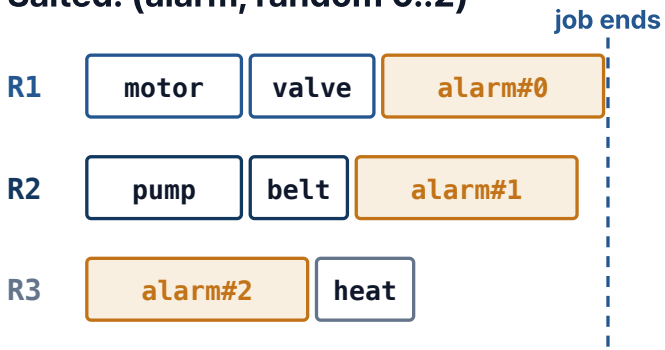
One hot key makes one task the bottleneck while the others sit idle

Hash by key



one reducer reads every alarm value

Salted: (alarm, random 0..2)



if the partitioner spreads them, a small pass sums the parts

Remedy: for counts, salt the hot key, count the parts, then sum the partial counts

Salting divides the hot key into **partial counts** before a final merge

MapReduce Summary

Simple and robust, at the cost of disk between steps

Simple to write

two functions:
map and reduce

Scales out

the system spreads tasks
and retries failures

Slow between steps

every intermediate
result goes to disk

The orange card is **the price** of fault tolerance

Issues with MapReduce

Multi-step queries become chains of jobs that round-trip through disk

MapReduce



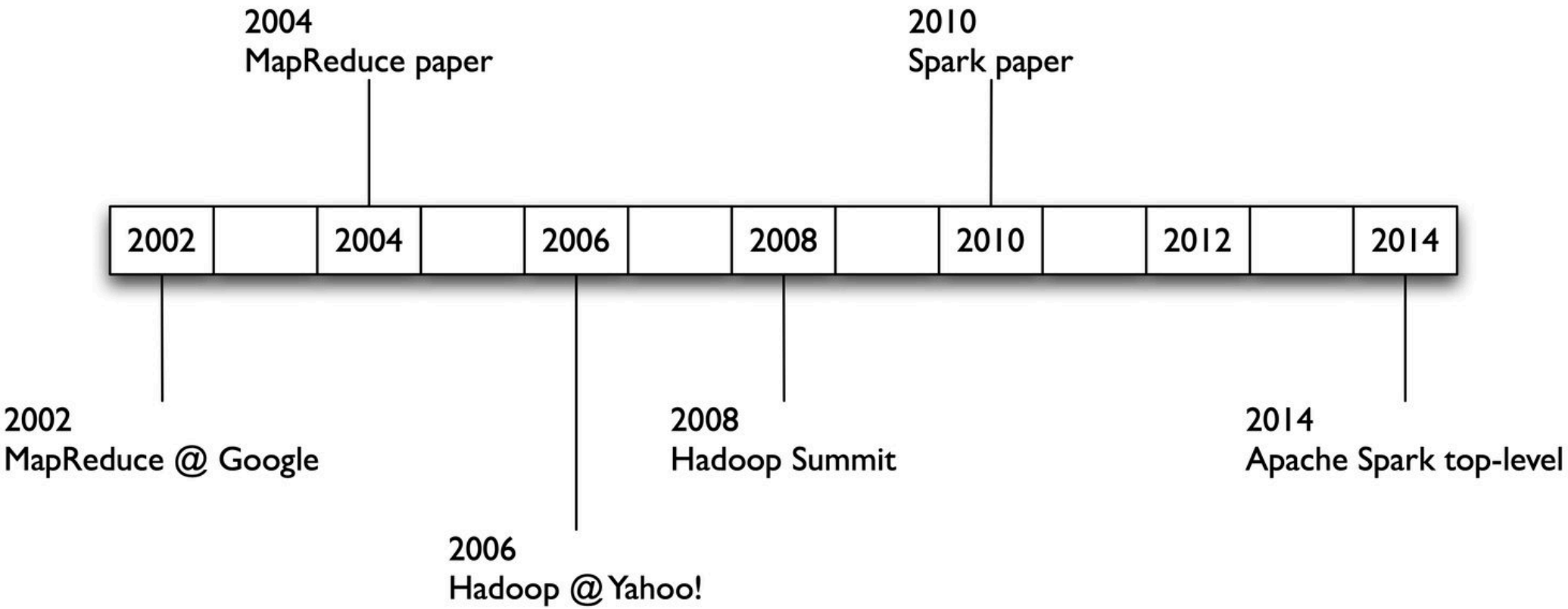
A query that needs three steps becomes **three jobs**, each writing all of its output to disk

Each job **writes everything** before the next can start

02 · How Spark Runs a Job

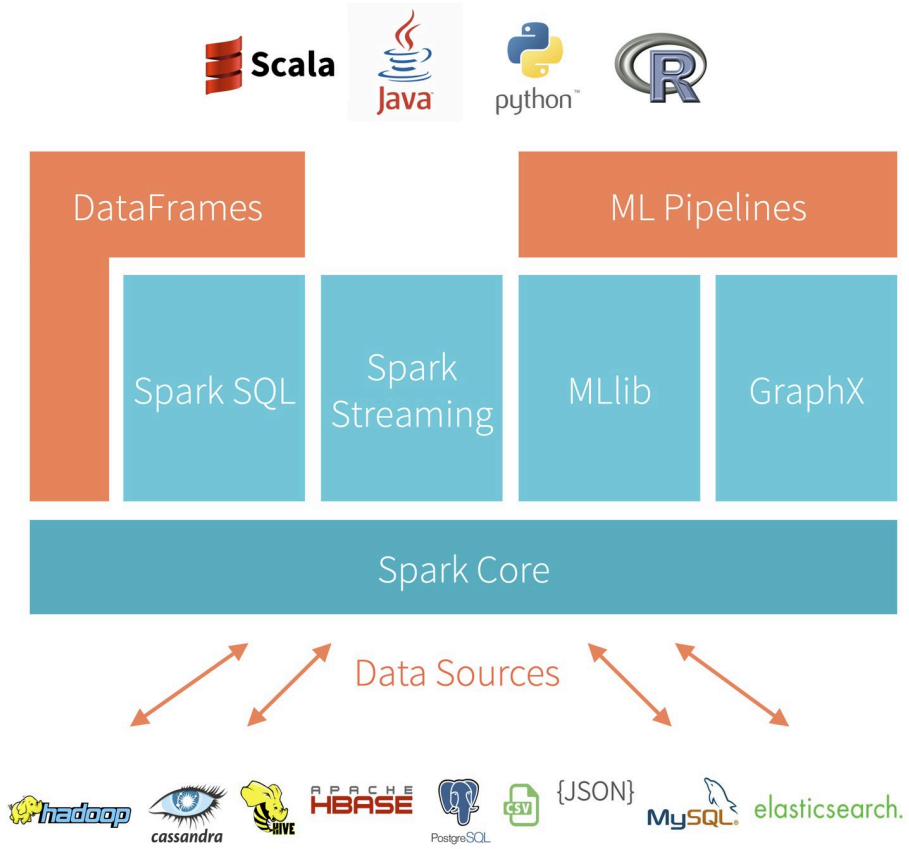
01	Processing on Many Machines	split a large job across machines
02	**How Spark Runs a Job**	understand when and where work runs
03	Working with Spark	query data and save the results

Spark: A Brief History



Six years from MapReduce to Spark, four more to Apache top level

Spark APIs



One core, four libraries, one DataFrame API on top

Why Spark? – Programmability

WordCount in 50+ lines of Java MR

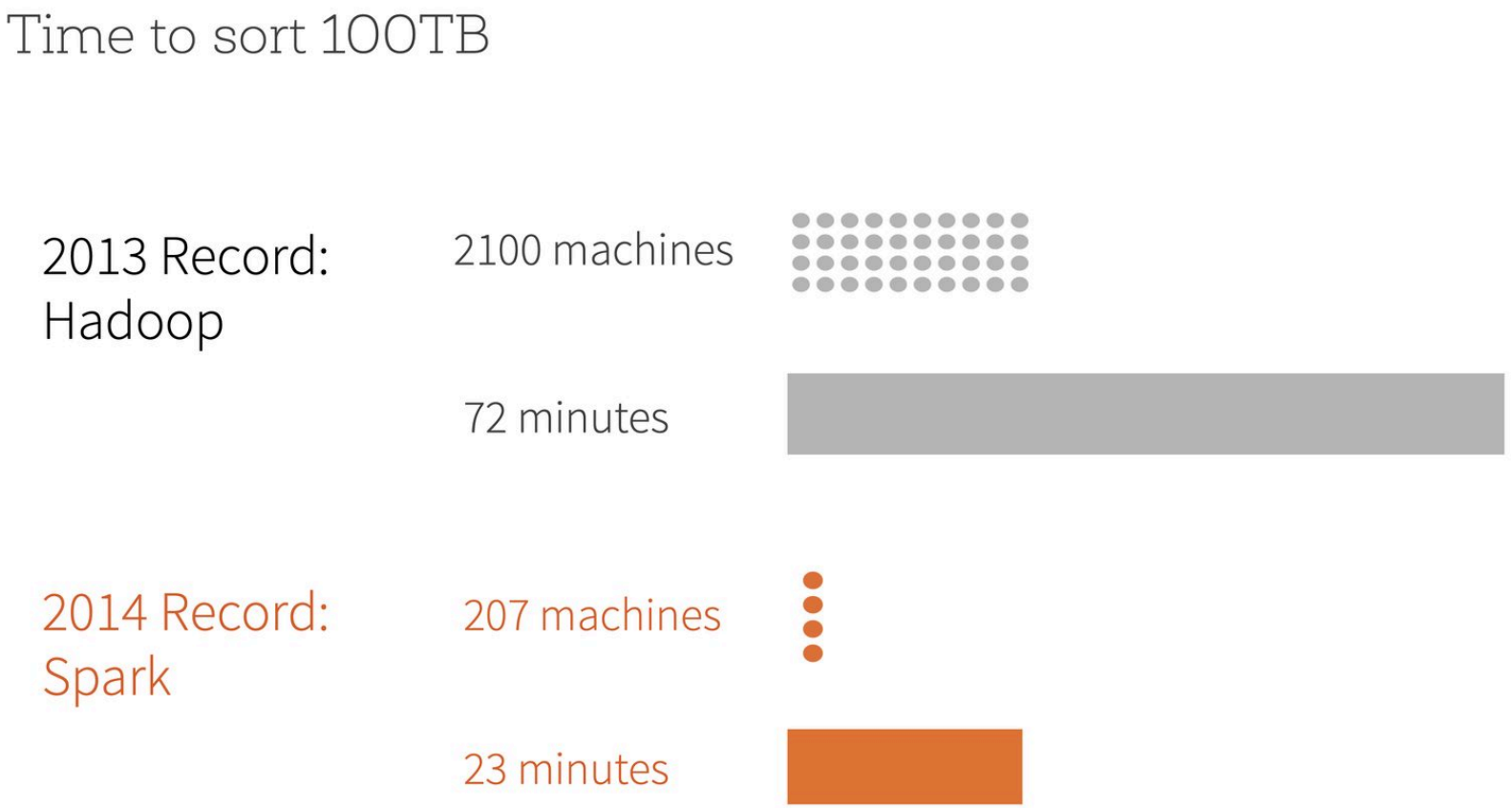
```
public class WordCount {
  public static class TokenizerMapper
       extends Mapper<Object, Text, Text,
IntWritable> { ... }
  public static class IntSumReducer
       extends Reducer<Text, IntWritable,
Text, IntWritable> { ... }
  public static void main(String[] args) {
    Job job = Job.getInstance(conf, "word
count");

job.setMapperClass(TokenizerMapper.class);
...
  }
}
```

WordCount in 3 lines of Spark

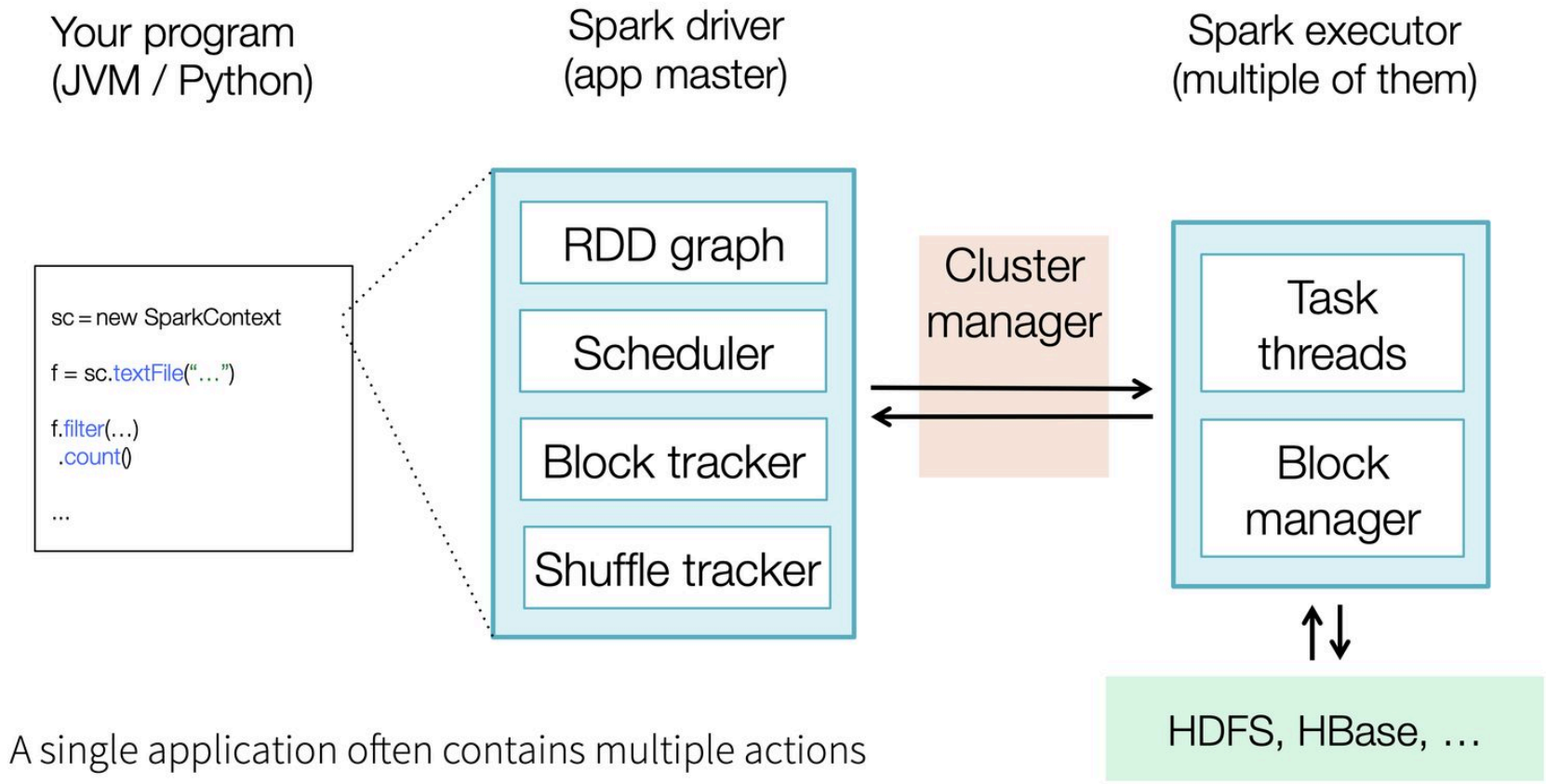
```
val f = sc.textFile(inputPath)
val w = f.flatMap(l => l.split("
")).map(word => (word, 1)).cache()
w.reduceByKey(_ +
_).saveAsTextFile(outputPath)
```

Why Spark? – Performance



Historical GraySort records: **2013 Hadoop** versus **2014 Spark**

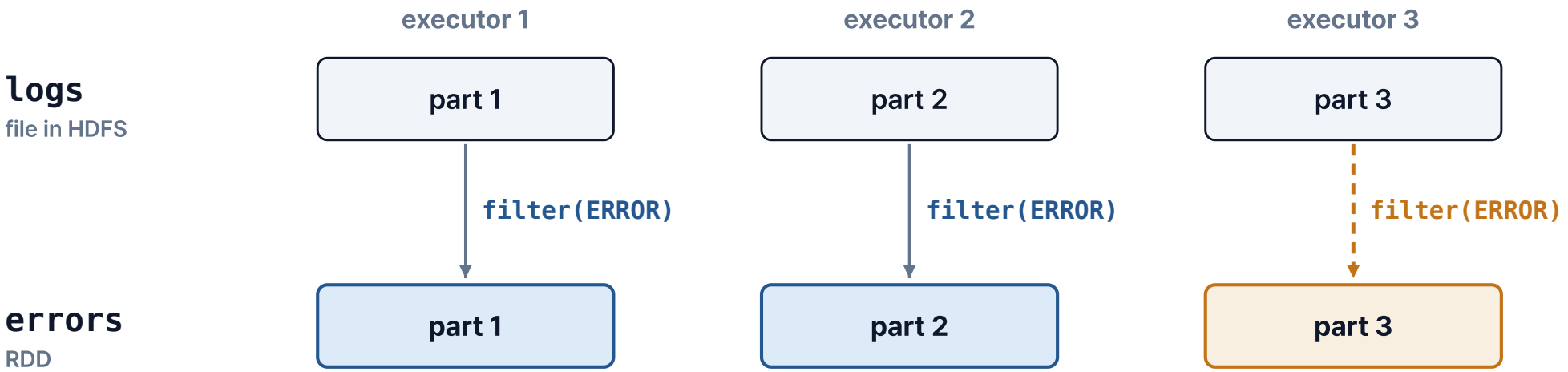
Driver and Executors



Driver plans, cluster manager hands out, **executors run and hold the blocks**

RDDs: Partitions and Lineage

An RDD is partitions spread over executors plus the recipe that made them



Executor 3 loses its part: Spark replays **filter** on that part only, using the lineage

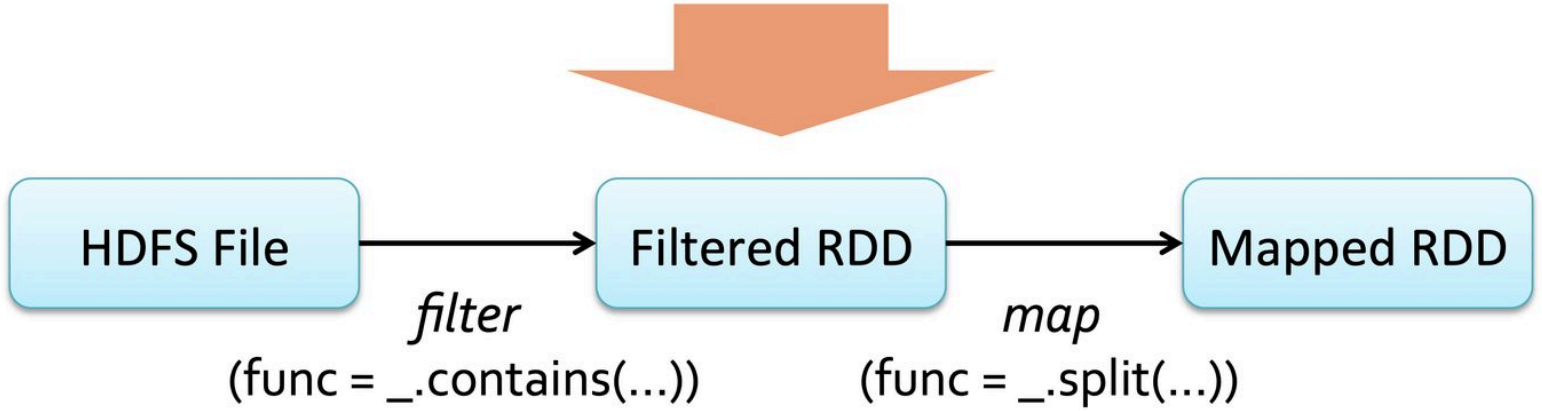
Recovery replays the recipe for **the lost partition only**

RDD Lineage

RDDs track *lineage* information that can be used to efficiently reconstruct lost partitions

Ex:

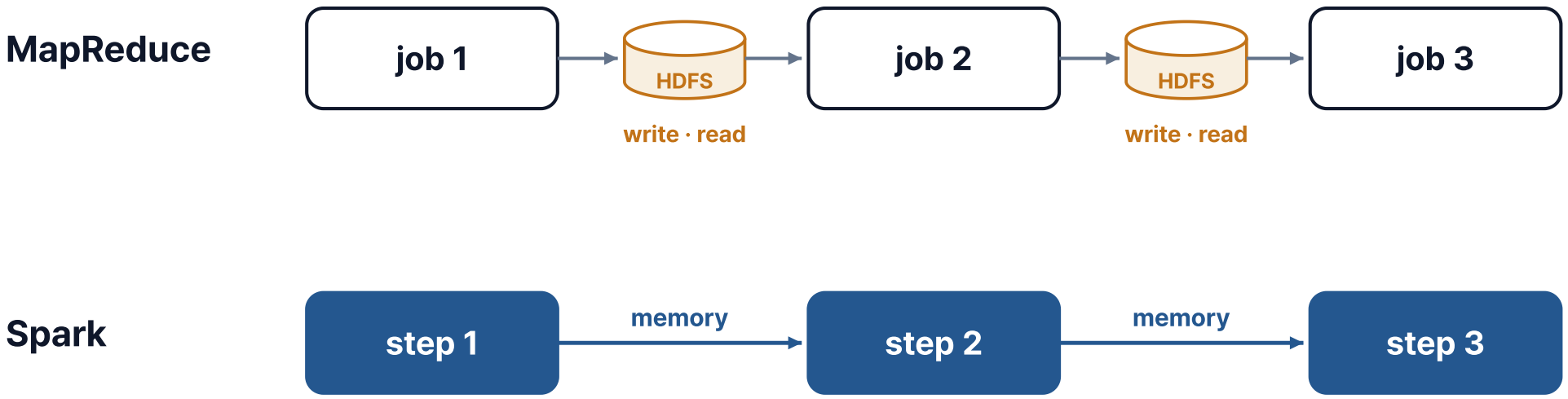
```
messages = textFile(...).filter(_.startsWith("ERROR"))
                        .map(_.split('\t')(2))
```



Each RDD remembers **the function that made it** from its parent

MapReduce vs RDD

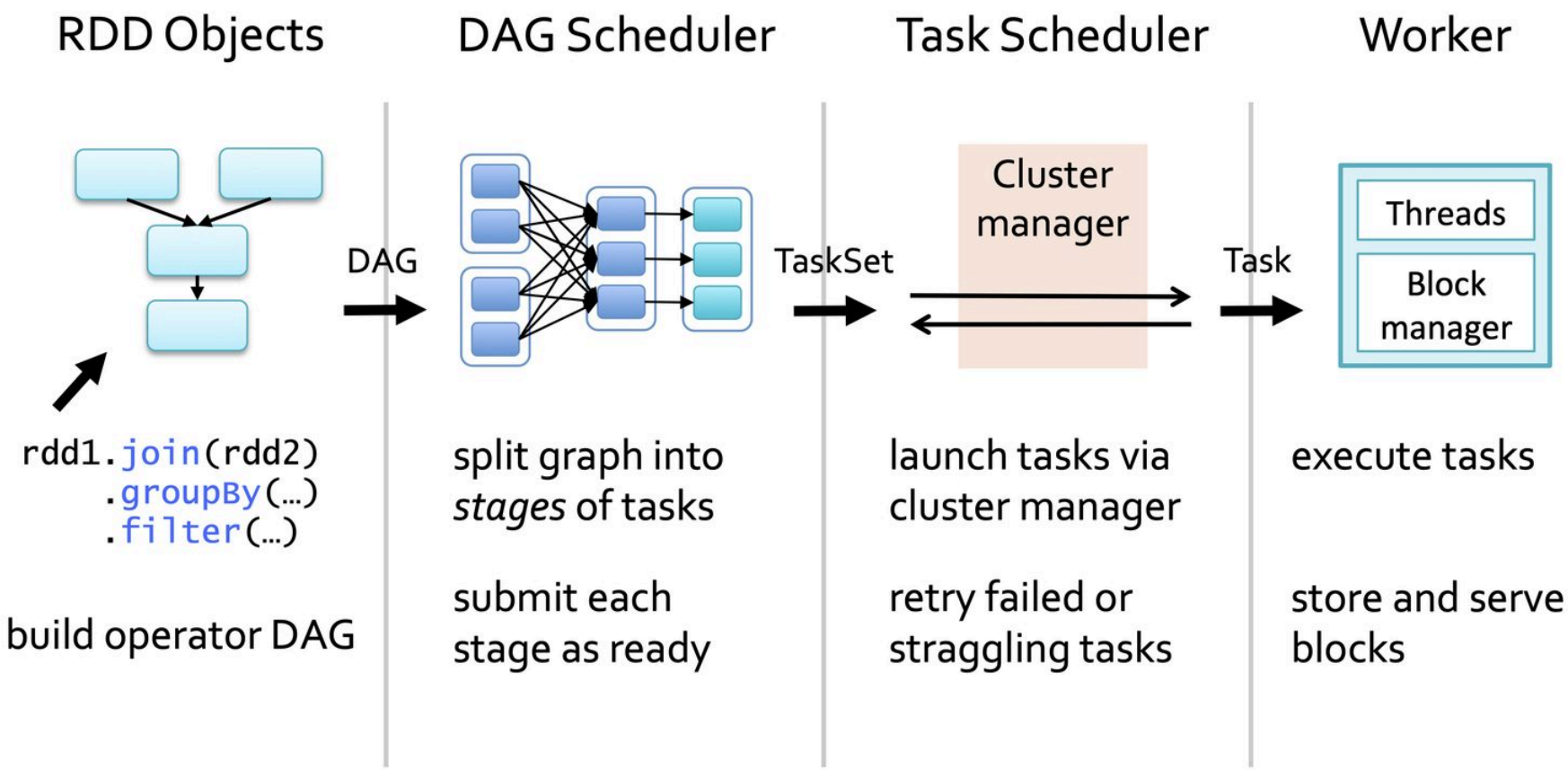
Same three steps: MapReduce stores between jobs, Spark chains them in one plan



Spark keeps intermediate partitions in memory when it can and rebuilds lost ones from lineage

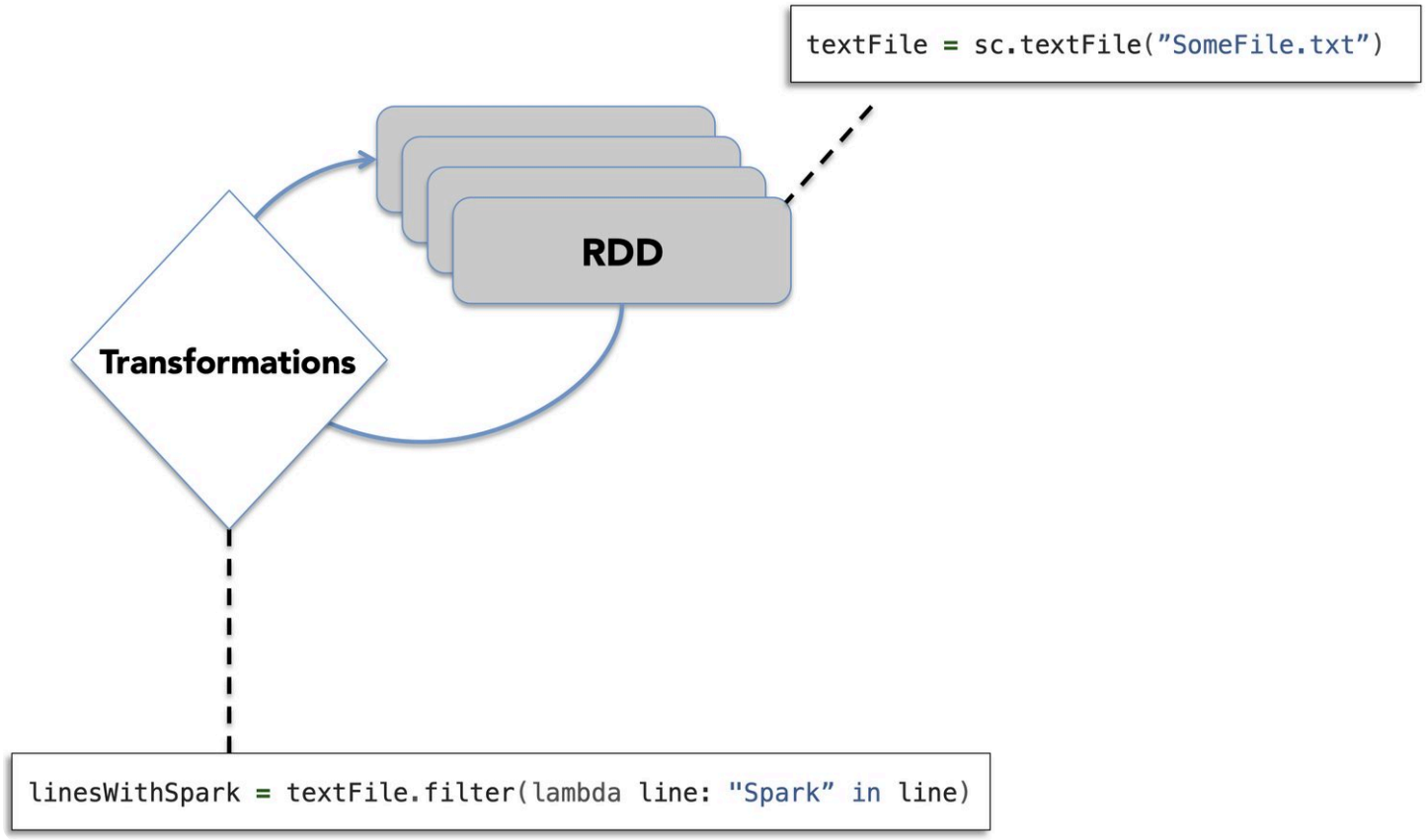
Disk between every job versus **memory within one plan**

RDD Execution Process



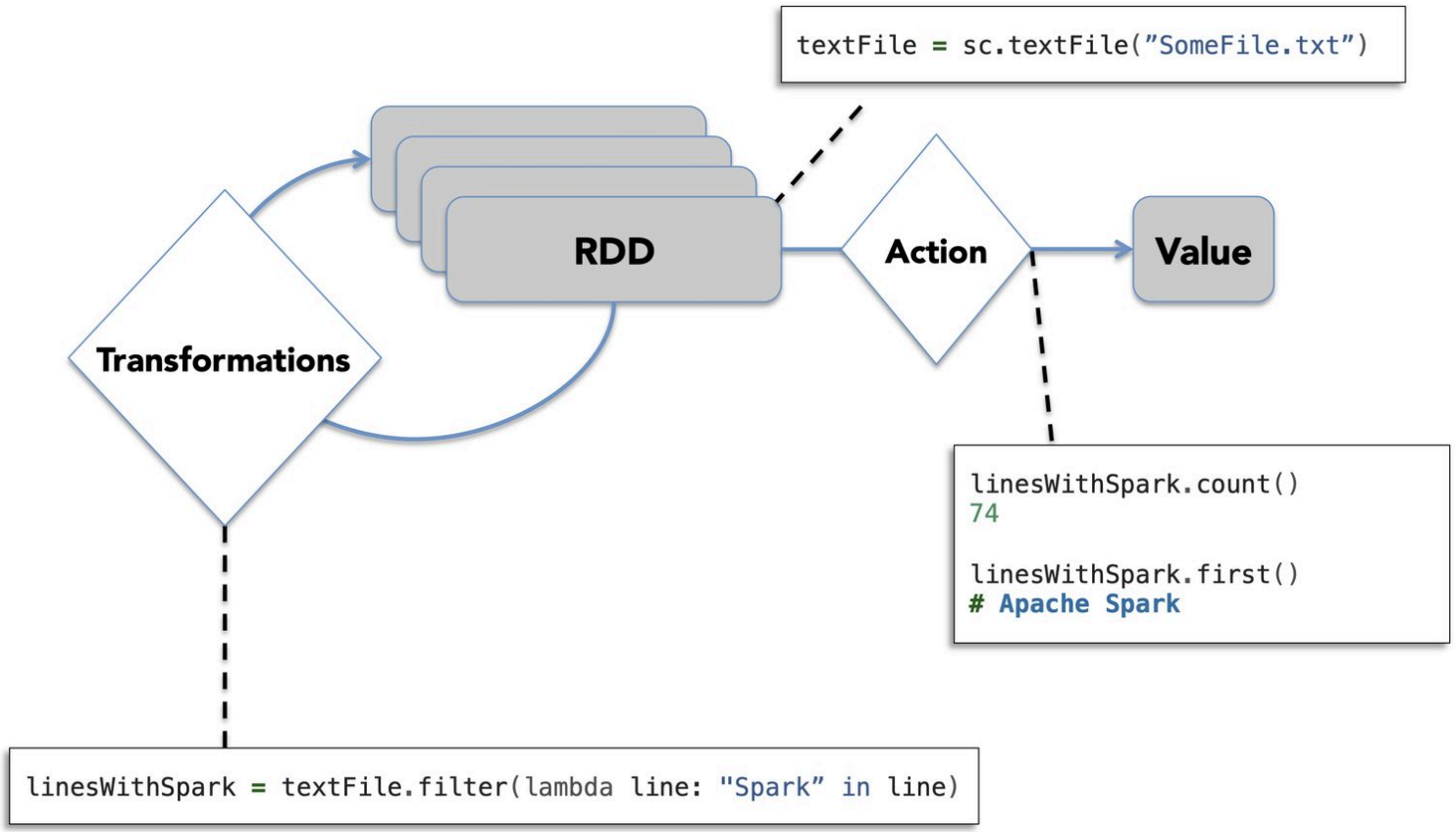
Code → DAG → stages → tasks → workers

RDD Transformations



Transformations go from RDD to RDD; **nothing has run yet**

RDD Actions



An action turns the tree into a value; **that is when work happens**

Map and Filter in PySpark

```
>>> rdd = sc.parallelize([1, 2, 3, 4])
>>> rdd.map(lambda x: x * 2).collect()
[2, 4, 6, 8]

>>> rdd.filter(lambda x: x % 2 == 0).collect()
[2, 4]

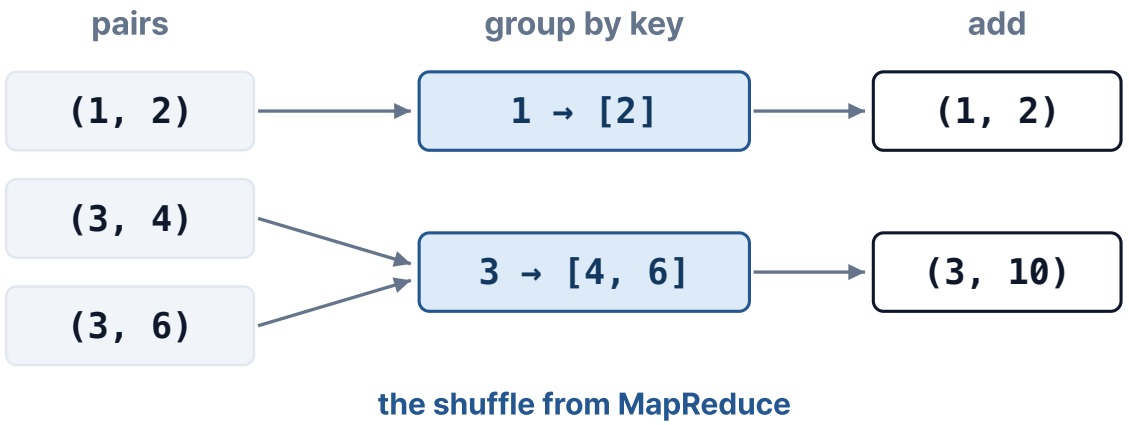
>>> rdd2 = sc.parallelize([1, 4, 2, 2, 3])
>>> sorted(rdd2.distinct().collect())
[1, 2, 3, 4]
```

Function literals (lambdas) are closures automatically passed to workers

Key Aggregation in PySpark

reduceByKey groups pairs by key, then combines the values

```
>>> rdd = sc.parallelize([(1, 2), (3, 4), (3, 6)])
>>> sorted(rdd.reduceByKey(lambda a, b: a + b).collect())
[(1, 2), (3, 10)]
```



The grouping step is **a shuffle**, just as in MapReduce

Caching Reused Data

```
lines = s.read().textFile("hdfs://logfile.log");
errors = lines.filter(l->l.startsWith("ERROR"));
sqlerrors = errors.filter(l->l.contains("sqlite"));
sqlerrors.collect();
```

If any server fails before the end, then Spark must restart




```
lines = s.read().textFile("hdfs://logfile.log");
errors = lines.filter(l->l.startsWith("ERROR"));
errors.persist();
sqlerrors = errors.filter(l->l.contains("sqlite"));
sqlerrors.collect()
```

Spark can recompute the result from errors



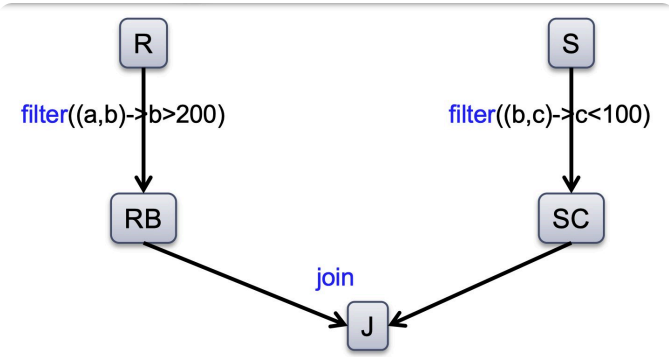
persist() keeps the middle RDD; recovery and reuse start **from there**

From SQL to RDD Code

```
R(A,B)
S(A,C)
```

```
SELECT count(*) FROM R, S
WHERE  R.B > 200 AND S.C < 100 AND R.A = S.A
```

```
def pair(line):
    a, value = line.split(",")
    return int(a), int(value)
R = sc.textFile("R.csv").map(pair)
S = sc.textFile("S.csv").map(pair)
RB = R.filter(lambda t: t[1] > 200)
SC = S.filter(lambda t: t[1] < 100)
J = RB.join(SC)
J.count()  # action
```



The lineage is a relational algebra tree, but **you ordered the operators**

03 · Working with Spark

01	Processing on Many Machines	split a large job across machines
02	How Spark Runs a Job	understand when and where work runs
03	**Working with Spark**	query data and save the results

DataFrames in Spark

RDD of Python objects

```
User(…)
User(…)
User(…)
User(…)
```

Spark sees opaque objects

DataFrame

name	country	age
Ana	Canada	31
Min	Korea	27
Leo	Canada	45

Spark knows **column names and types**

Knowing the columns lets Spark **optimize the query**

DataFrame Operations

DataFrames hold rows with a known schema and offer relational operations on them through a DSL

```
val users = spark.sql("select * from users")
val massUsers = users('country === "Canada")   // expression AST, not a closure
massUsers.count()
massUsers.groupBy("name").avg("age")
```

`'country === "Canada"` builds an **expression tree** Spark can read; a lambda in `filter` is opaque to the engine.

Spark SQL: One Query, Two Ways

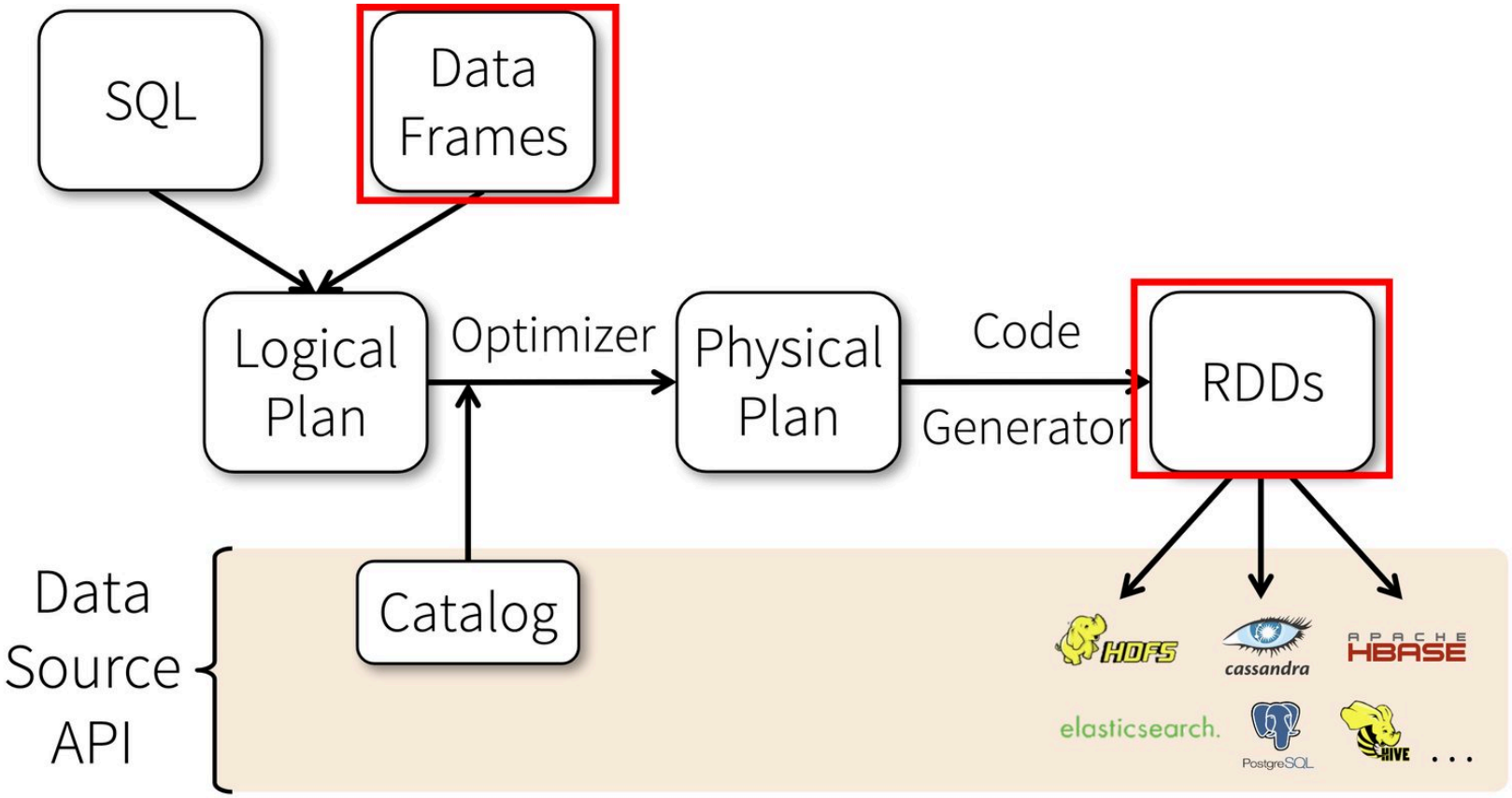
```python
from pyspark.sql import functions as F

sales = spark.read.parquet("sales/")
(sales.filter(sales.region == "Asia")
      .groupBy("product")
      .agg(F.sum("amount").alias("total"))
      .orderBy(F.desc("total")))
```

```python
sales.createOrReplaceTempView("sales")
spark.sql("""
  SELECT product, SUM(amount) AS total
  FROM sales WHERE region = 'Asia'
  GROUP BY product ORDER BY total DESC""")
```

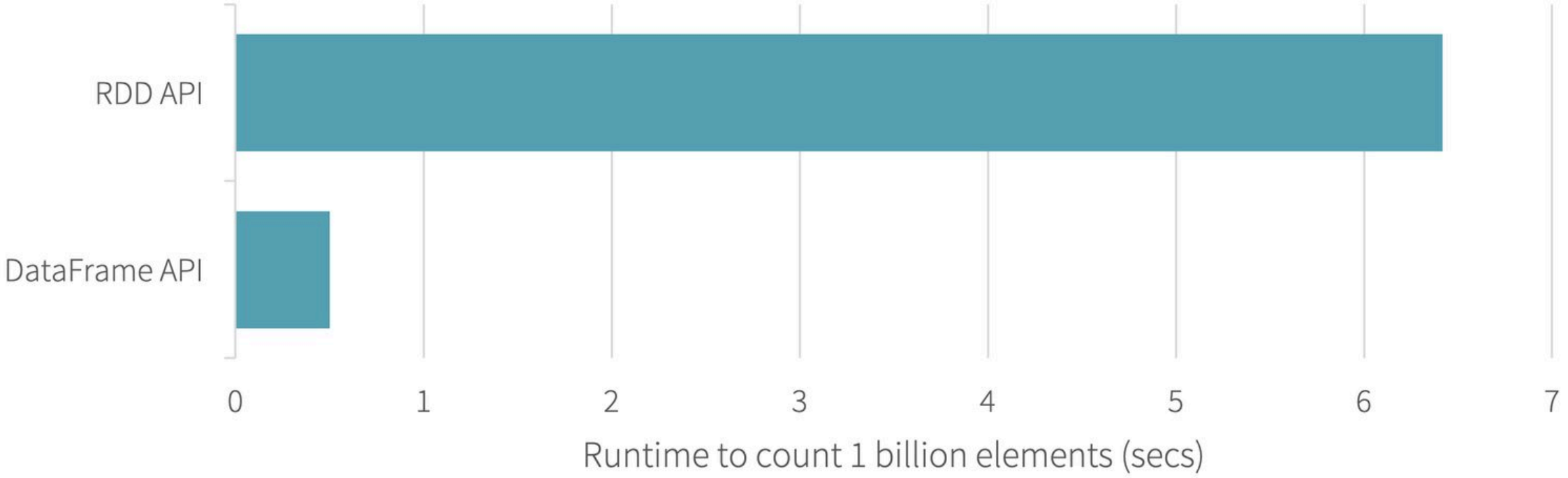
Both go through the **same optimizer** and run as the same plan

The Optimized Plan



SQL and DataFrames meet in a logical plan; **the optimizer is back**

DataFrame vs RDD Speed



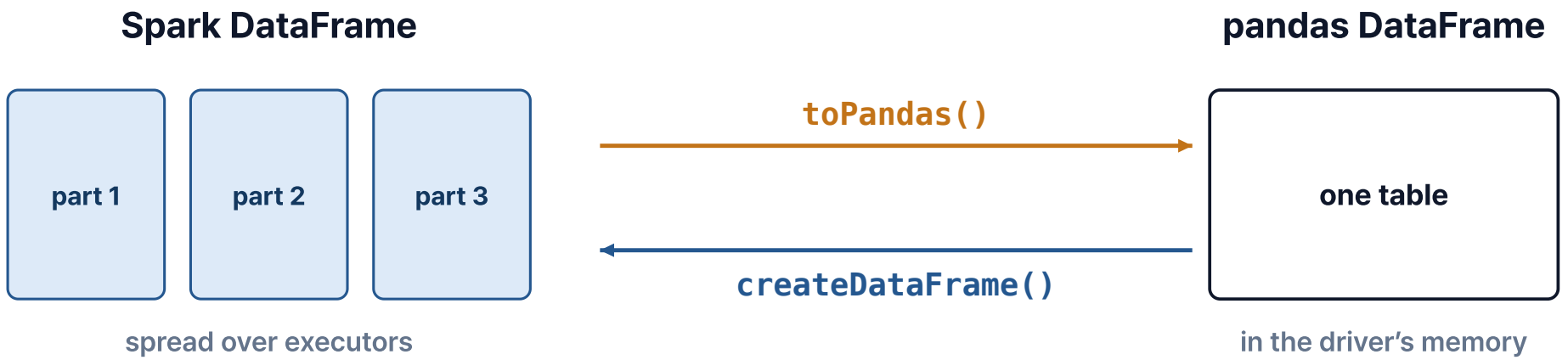
Historical example: counting **one billion elements** with RDD and DataFrame APIs

RDD vs DataFrame

Aspect	RDD	DataFrame
Level	Low-level API (manual control)	High-level API (table abstraction)
Optimization	No SQL expression optimizer	Catalyst optimizer (automatic)
Execution	User functions over objects	Optimized SQL expressions and generated code
Schema	Unstructured	Schema-aware
Ease of use	Complex functional code	Simple SQL-like syntax
When to use	Custom logic, unstructured data	ETL, analytics, ML pipelines

DataFrames and pandas

toPandas() moves every partition into the driver



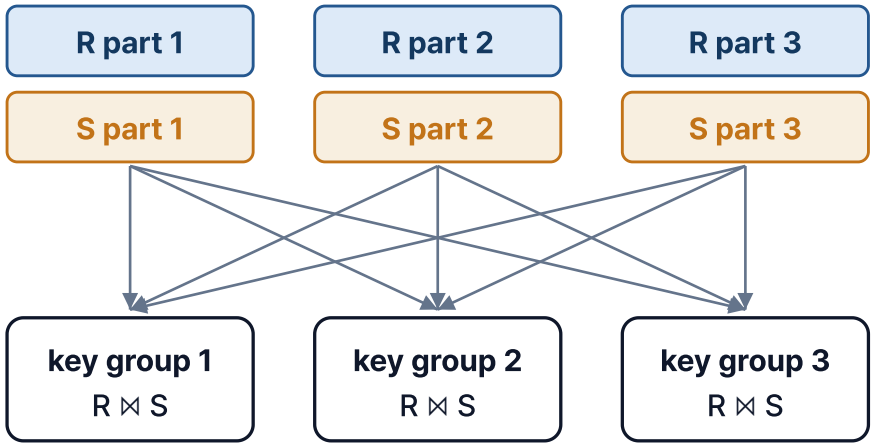

```
pandas_df = spark_df.toPandas()                # small results only
spark_df = spark.createDataFrame(pandas_df)
```

Convert to pandas **after** the result is small

DataFrame Joins

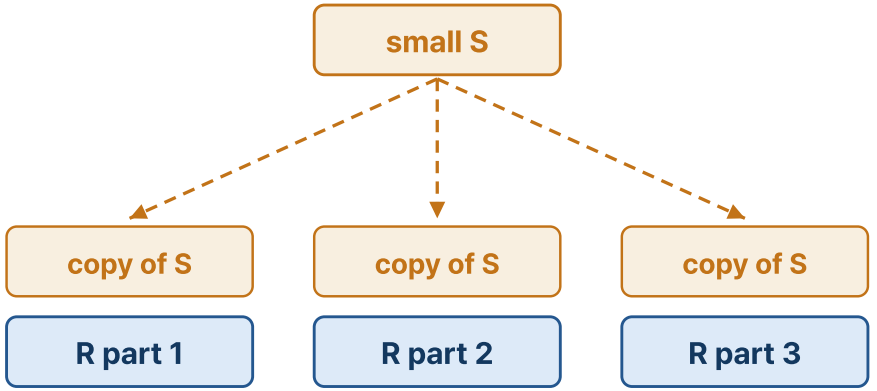
Two ways to bring matching keys together

Shuffle join: both tables large



every row may cross the network

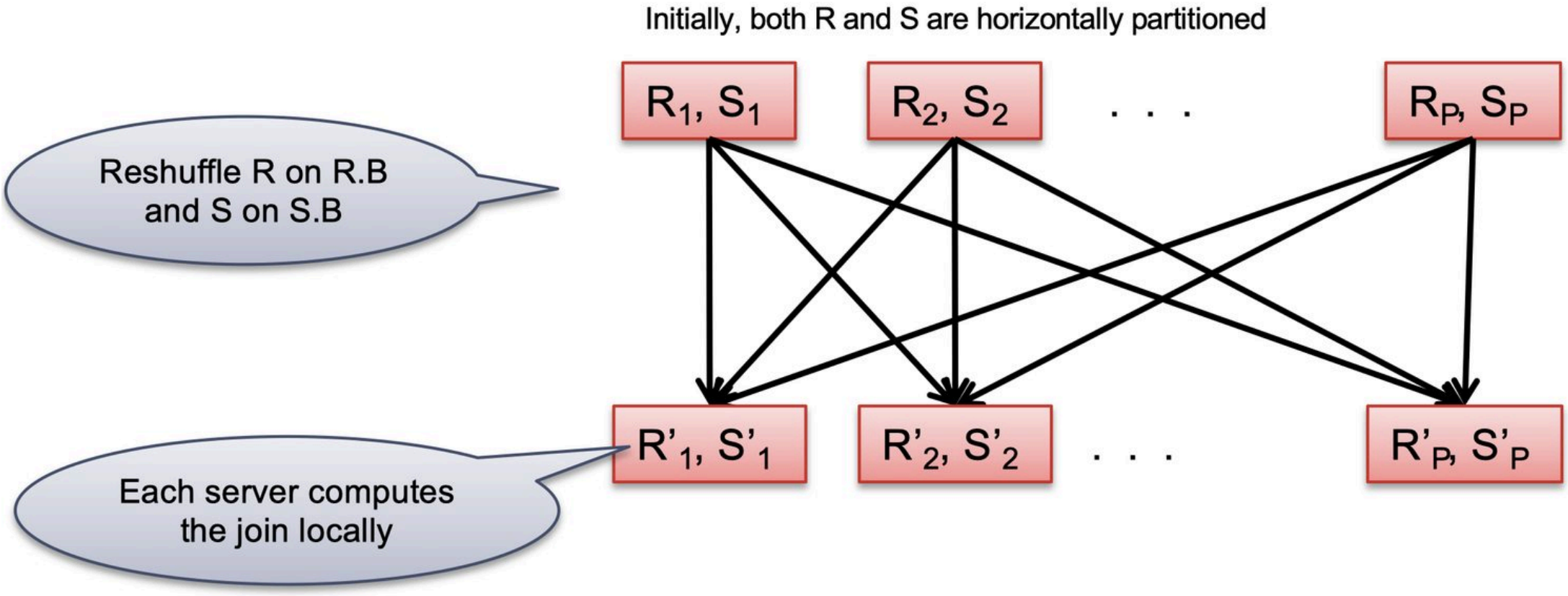
Broadcast join: one table small



the large table stays where it is

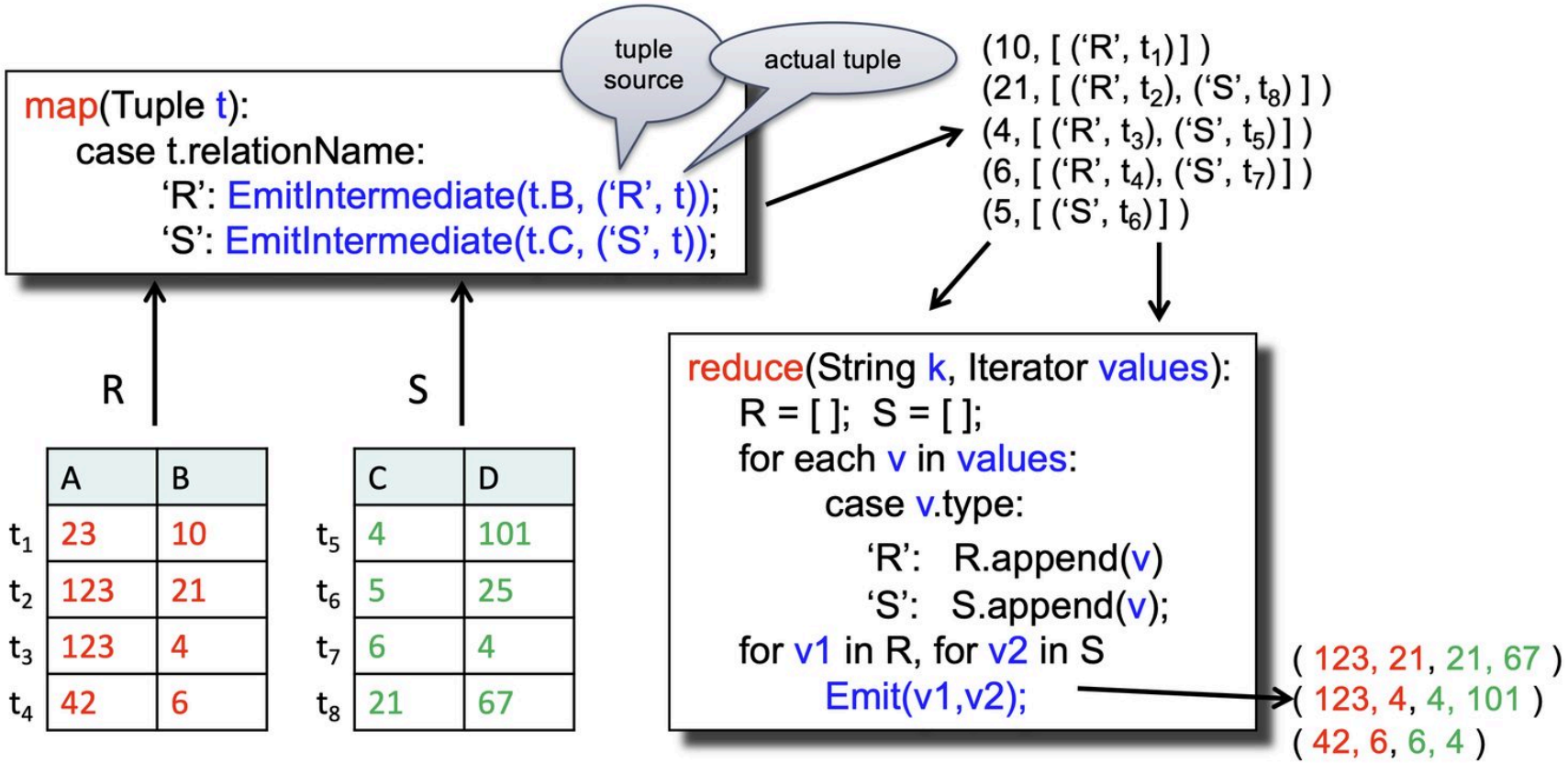
Move both tables by key, or **copy the small one** everywhere

Shuffle Join



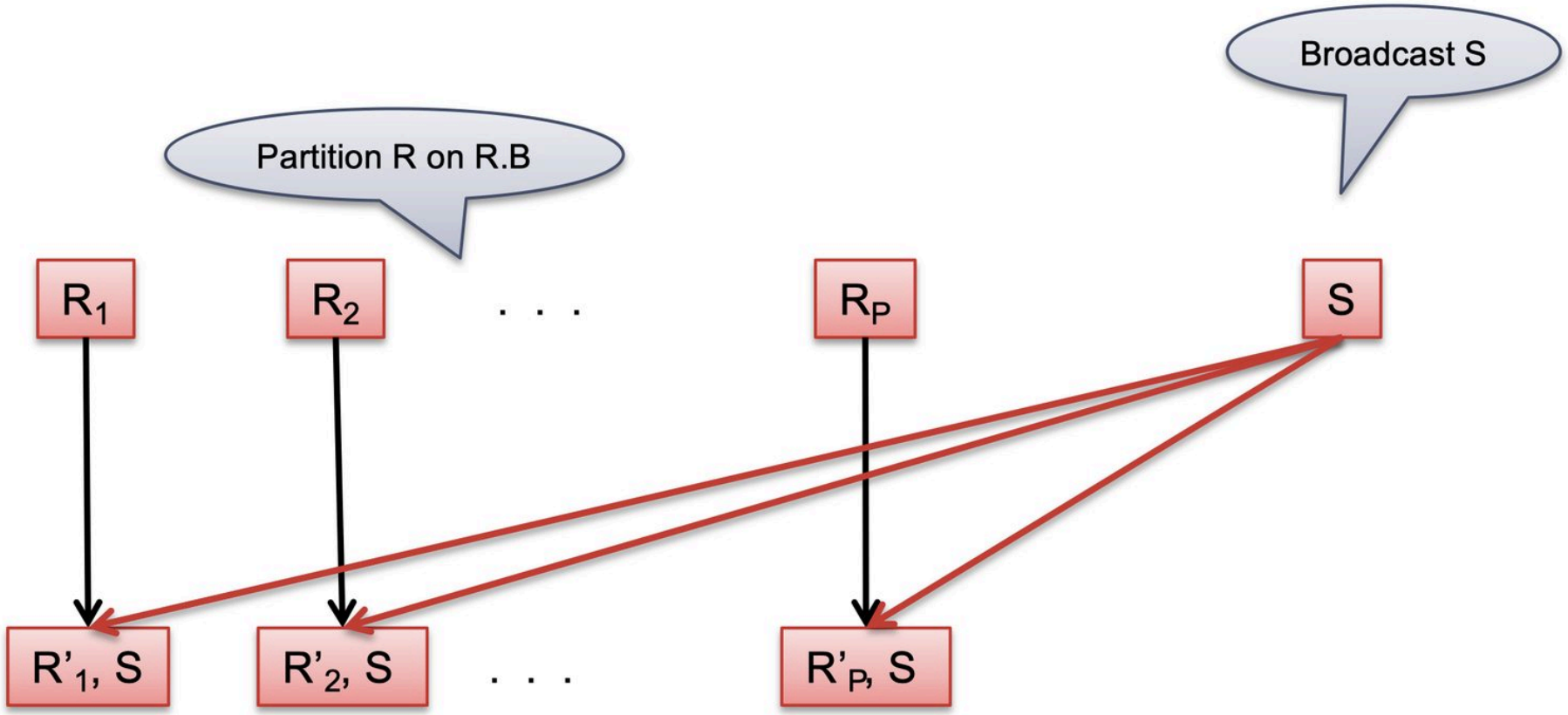
Reshuffle both sides on the join key, then **each server joins locally**

Shuffle Join by Key



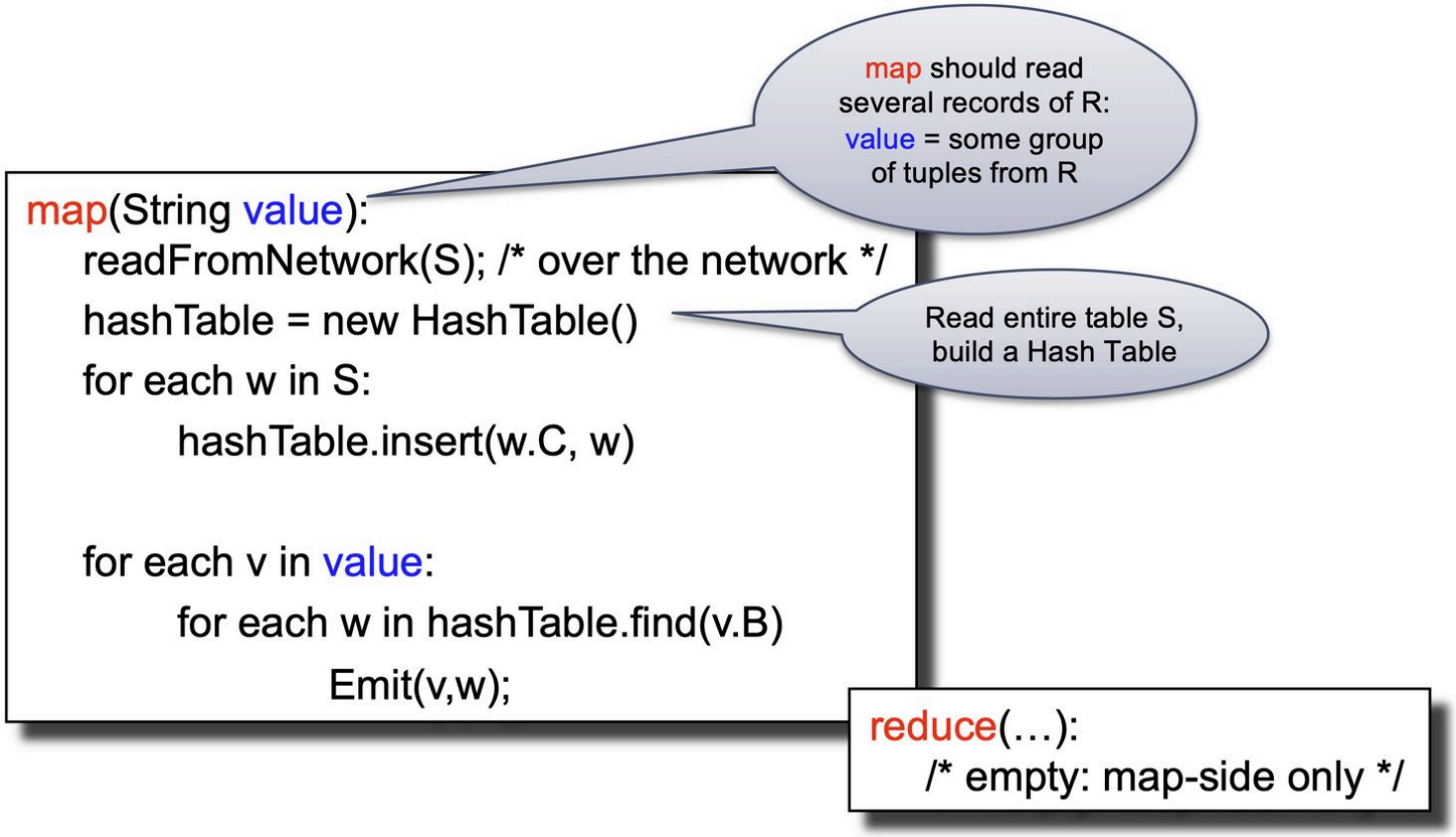
Tag each tuple with its source; the reduce **pairs R with S inside one key**

Broadcast Join



Broadcast the **small** table; the large input can keep its existing partitions

Broadcast Join in Each Worker

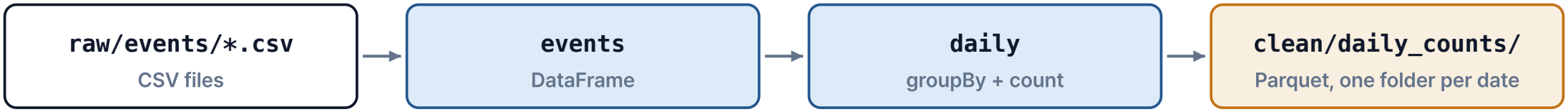


A hash table of S in every mapper; **no shuffle, no reduce**

Comparison of Join Algorithms

	Partitioned Hash Join	Broadcast Join
When to use	Both tables are large, or not already partitioned by the join key	One table is very small (MBs to tens of MBs) and can be replicated to all workers
Data scale	Large ↔ Large (fact–fact join)	Large ↔ Small (fact–dimension join)
Data movement	Both tables are shuffled across the network	Only the small table is sent to all nodes
Performance	Shuffle cost depends on size, partitioning and skew	Avoids shuffling the large input; broadcast and memory costs remain
Limitations	High shuffle cost, longer execution time	Small-table size limit, risk of out-of-memory
Rule of thumb	If both are big, partition	If one is small, broadcast

Files In, Files Out




```
events = spark.read.csv("raw/events/*.csv", header=True, inferSchema=True)
daily = events.groupBy("date", "machine").count()

(daily.write.mode("overwrite")
      .partitionBy("date")
      .parquet("clean/daily_counts/"))
```

Read raw files, transform, write **Parquet**; the write is the action that runs it all

Where Spark Runs

Where	Examples	Use it when
Your laptop	pip install pyspark, master local[*]	learning, testing on a sample
Managed platform	Databricks, AWS EMR, Google Dataproc	the data does not fit on one machine
Not Spark at all	pandas, Polars, DuckDB	the data fits on one machine

If the data **fits on one machine**, you do not need Spark

Takeaway: Big Data I

1. When data outgrows one machine, **map → shuffle → reduce** divides the work; a hot key can still leave one slow task
2. Spark uses **lineage** to recover lost partitions and can cache reused data
3. Write **DataFrames or SQL**; an action runs the lazy plan, and joins often pay for shuffles